

计算机科学与技术学院神经网络与深度学习课程实验报告

实验题目：LeNet 实现 MNIST 图像分类		学号：202300130183
日期：2025/10/16	班级：2023 级智能班	姓名：宋浩宇
Email: zhitian0420@gmail		
实验目的： 1.理解卷积神经网络（CNN）的基本原理和结构，特别是 LeNet 的网络架构。 2.掌握使用 LeNet 网络对 MNIST 数据集进行图像分类的方法。 3.学习如何评估卷积神经网络的分类性能，并调整网络参数以优化性能。 4.掌握深度学习模型训练、验证和测试的完整流程。 5.理解超参数调优对模型性能的影响，掌握模型优化的基本方法。		
实验软件和硬件环境： 软件环境： 主系统：Windows 11 家庭中文版 23H2 ; ArchLinux 编辑器：Visual Studio Code ; PyCharm 2025.1.2 Python 解释器：Python 3.9 Pytorch 版本：2.8.0 CUDA 版本：12.8.97 硬件环境： CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz 内存：32.0 GB (31.6 GB 可用) 磁盘驱动器：NVMe WD_BLACKSN850X2000GB 显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU CUDA 核心数：7424		
实验原理和方法： 实验原理为 LeNet，方法为卷积-池化-全连接构建模型，超参数调优。		
实验步骤：（不要求罗列完整源代码） 1. 导入数据集以及数据预处理 <pre>import numpy as np from tqdm import tqdm from torchvision import datasets, transforms import torch MNIST_PATH = r'F:\DATASET\MNIST\mnist_dataset_dir' transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))]) batch_size = 64 train_data = datasets.MNIST(root=MNIST_PATH,</pre>		

```

        train=True,
        transform=transform,
        download=True)
test_data = datasets.MNIST(root=MNIST_PATH,
                           train=False,
                           transform=transform,
                           download=True)
train_gen = torch.utils.data.DataLoader(train_data,
                                         batch_size=batch_size,
                                         shuffle=True)
test_gen = torch.utils.data.DataLoader(test_data,
                                       batch_size=batch_size,
                                       shuffle=False)

```

由于我本地已经下载好了 MNIST 数据集，所以可以节省不少时间，数据集导入后转化为张量，并进行正则化。

2. 构建网络

```

class LeNet(torch.nn.Module):
    def __init__(self, num_classes=10):
        super(LeNet, self).__init__()
        self.conv1 = torch.nn.Conv2d(kernel_size=5,
padding=2,in_channels=1, out_channels=6)
        self.s2 = torch.nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv3 = torch.nn.Conv2d(kernel_size=5, padding=0,
in_channels=6, out_channels=16)
        self.s4 = torch.nn.AvgPool2d(kernel_size=2, stride=2)
        self.fc1 = torch.nn.Linear(in_features=16 * 5 * 5,
out_features=120)
        self.fc2 = torch.nn.Linear(in_features=120, out_features=84)
        self.fc3 = torch.nn.Linear(in_features=84,
out_features=num_classes)
        self.get_layer_info(torch.randn(1, 28, 28))
    def get_layer_info(self, x):
        x = self.conv1(x)
        print(f"Conv1 output shape: {x.shape}")
        x = torch.nn.functional.relu(x)
        print(f"ReLU1 output shape: {x.shape}")
        x = self.s2(x)
        print(f"AvgPool1 output shape: {x.shape}")
        x = self.conv3(x)
        print(f"Conv2 output shape: {x.shape}")
        x = torch.nn.functional.relu(x)
        print(f"ReLU2 output shape: {x.shape}")
        x = self.s4(x)
        print(f"AvgPool2 output shape: {x.shape}")

```

```

        x = torch.flatten(x)
        print(f"Flatten output shape: {x.shape}")
        x = self.fc1(x)
        print(f"FC1 output shape: {x.shape}")
        x = torch.nn.functional.relu(x)
        print(f"ReLU3 output shape: {x.shape}")
        x = self.fc2(x)
        print(f"FC2 output shape: {x.shape}")
        x = torch.nn.functional.relu(x)
        print(f"ReLU4 output shape: {x.shape}")
        x = self.fc3(x)
        print(f"FC3 output shape: {x.shape}")
        return x

    def forward(self, x):
        x = self.conv1(x)
        x = torch.nn.functional.relu(x)
        x = self.s2(x)
        x = self.conv3(x)
        x = torch.nn.functional.relu(x)
        x = self.s4(x)
        x = torch.flatten(x, 1)
        x = self.fc1(x)
        x = torch.nn.functional.relu(x)
        x = self.fc2(x)
        x = torch.nn.functional.relu(x)
        x = self.fc3(x)
        return x

```

我们顺便设置一个获取维度的函数，便于调试。

然后设置好损失和函数和优化器

```

model = LeNet()

criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

```

损失函数使用交叉熵损失函数，优化器使用 adam 优化器

3. 训练

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
print(f"训练设备: {device}")

for epoch in range(10):
    running_loss = 0.0
    for i, data in enumerate(tqdm(train_gen), 0):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

```

```

optimizer.zero_grad()
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
running_loss += loss.item()
print(f"Epoch {epoch+1} loss: {running_loss/len(train_gen)}")

```

用 cuda 训练更快

4. 测试

```

correct = 0
total = 0
model.eval()
for i, data in enumerate(tqdm(test_gen), 0):
    inputs, labels = data
    inputs, labels = inputs.to(device), labels.to(device)
    outputs = model(inputs)
    _, predicted = torch.max(outputs.data, 1)
    total += labels.size(0)
    correct += (predicted == labels).sum().item()

print(f"Accuracy of the network on the 10000 test images: {100 * correct / total}%")

```

结论分析与体会：

我们训练出来的模型的测试结果为：

```

0%|          | 0/157 [00:00<?, ?it/s]Epoch 10 loss: 0.018509948178929692
100%|██████████| 157/157 [00:01<00:00, 147.49it/s]
Accuracy of the network on the 10000 test images: 99.12%

```

从准确率上来看，我们的模型性能良好，效果很不错。

体会主要是，pytorch 真好用，使得搭建网络和训练的难度变得特别特别低。

就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1—3 道问答题：

问题 1： 为什么 lenet 要这样写，他的思路是什么，还是只是经验结论？

回答 1： LeNet 模型通过层层卷积和池化提取图像从局部到整体的特征，再经全连接层完成分类。其设计思想源于人类视觉系统的分层感知机制，既减少参数，又提高识别的平移不变性，其思路来源于脑神经科学的研究，并非凭空产生或从经验总结。